

to-do:

1. Taylor series of different types of functions such as sigmoid vs exp vs relu. Look at the falloff of coefficients.
2. Taylor expansion of linear attention like mamba vs softmax attention
3. Does the complexity increase/decrease with RoPE positional encodings. How does the complexity look with/without RoPE?

Properties of SM attention:

1. Q works in the double dual space of V, the dual space of Q
2. K works in the dual space of V
3. The attention matrix and V create an inner product space on S
4. Q and K create an inner product space on d
5. There is a recurrence relation on S between K and V
 - a. This is due to the inner product of K and V
6. There is an independence of the queries across the sequence.
 - a. This is due to the exponential in SM.
7. SM attention is essentially an RNN at each timestep, not an RNN over all timesteps. This is due to the uniqueness of the queries.
 - a. The RNN is unique because of the exponential. Without the exponential, we could share RNNs across the entire matrix.
8. If $Q = (N, d)$, $K = (d, M)$, $V = (M, e)$, the second dimension of V (e) doesn't matter and the first dimension of Q (N) doesn't matter.
 - a. This means the vector space V^M has components where each component is in V^e . To make this easy to visualize, just make $e=1$ which is just as equivalent as setting e to any other value.

Tests:

1. Is SM needed?
 - a. YES! If SM is removed, then the independence goes away. Independence is forced by the SM function on the inner product space of d. We still have an inner product space on S by K and V, but no longer a function on the inner product space of d. This can be seen with cotattention doing worse than SM attention.
 - b. This also means that an inner product on S alone is not enough. We need more.
2. Is an inner product function on S equal to or better than one on d?
 - a. **TRY THIS**
3. Can Q be static?
 - a. No. If Q is static, then there is no independence among queries. This runs into lower performance issues.
 - b. https://wandb.ai/gmongaras1/Small_Attn/runs/ap2mrzbh?nw=nwusergmongaras
4. Can K be static and is this useful?
 - a. No. Surprisingly runs into the same problem as static Q. Gets the exact same performance.

- b. https://wandb.ai/gmongaras1/Small_Attn/runs/gfv2kahk
5. Is softmax necessary?
 - a. Can it be replaced with another row-wise function like norm? (just remove the exp)
 - i. If we replace the softmax function with $\text{Attn} = (\text{attn}^{**2})/(\text{attn}^{**2}).\text{sum}(-1)$, using the square instead of the exponential, we get about the same results as if we let Q and K be static. So slightly worse than softmax. This is essentially a dynamic base while SM uses a base of e.
https://wandb.ai/gmongaras1/Small_Attn/runs/9fdgalko?nw=nwusergmongaras
 - ii. If we replace the softmax function (base of e) with a learnable base initialized to 2: $\text{Attn} = (\text{base}^{**\text{weights}})/(\text{base}^{**\text{weights}}).\text{sum}(-1)$, then we do **as well as softmax**.
https://wandb.ai/gmongaras1/Small_Attn/runs/2gck7tdz?nw=nwusergmongaras
 - b. Can it be replaced with an element-wise function like ReLU?
 - i. No, the result is very similar to cosine attention.
6. What part of softmax is needed?
 - a. Is it the exponential?
 - i. Exponential seems to be needed. We can use other functions like sigmoid, but these are functions on the exponential.
 - b. Is it the normalization?
 - i. Test where the base is 0.5 and there is no division: $\text{Attn} = (0.5^{**\text{weights}})$.
 - ii. Yes norm is needed. The performance is quite a bit worse than normal.
7. Is a function on the inner product space on d alone or is the inner product space of S also needed?
 - a. IDK if this is possible.
8. Attention is an inner product space of exponentials:
 $e^{(Q^*K.T)}$
 If we write this out for each vector inner product, we have
 $e^{(q_1*k_1 + q_2*k_2 + \dots)}$
 Which is equivalent to
 $e^{(q_1*k_1)} * e^{(q_2*k_2)} * e^{(q_3*k_3)}$
 Which is a product between vectors???
- a. What if we replace the exponential of the inner product with just a product of the products
 $e^{(Q^*K.T)} = e^{(q_1*k_1)} * e^{(q_2*k_2)} * e^{(q_3*k_3)}$
 Instead we do $\rightarrow q_1*k_1 * q_2*k_2 * q_3*k_3$
 This creates the attention matrix not with inner products, but with products of vector products
 - b. Unfortunately this kind of works
9. If we replace the exponential function with sigmoid, does this work?
 - a. Sigmoid is $1/(1+e^{-x})$, which is still an exponential, so it turns sums into products.
 - b. It does, but not as good as exp. Likely cause the gradient sucks.

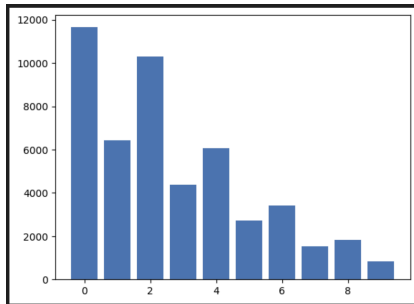
- c. https://wandb.ai/gmongaras1/Small_Attn/runs/su1yahb1?nw=nwusergmongaras
- 10. If we replace sigmoid with silu, it sucks. Like it's terrible.
 - a. `torch.nn.functional.silu(Q @ K.mT)`
 - b. I am guessing this is because we can get negative values
- 11. If we replace exp with sigmoid and do a product on the inner product space, we get really good results
 - a. `torch.nn.functional.sigmoid(queries.unsqueeze(-2) * keys.unsqueeze(-3)).prod(-1)`
 - b. https://wandb.ai/gmongaras1/Small_Attn/runs/z80ylv51?nw=nwusergmongaras
 - c. This seems to show that it's a nice nonlinear function on the product-product space.
 - d. Or maybe not since sigmoid is also an exponential function
- 12. Do we need both queries and keys or can we just do $Q @ Q.mT$ or $K @ K.mT$?
 - a. Just keys or just queries has variance explosion.
 - b. If we divide by the max value in the matrix (so that we don't have variance explosion), the model start better but ends worse
 - c. If we normalize the keys, the model performs significantly worse than normal
 - d. This indicates the uniqueness for each t, s pair is important
 - e. https://wandb.ai/gmongaras1/Small_Attn/runs/yv8wdauc?nw=nwusergmongaras
- 13. **Conclusions so far:**
 - a. Exponential on the inner product space is necessary. The base seems to not matter. the value of e is just a choice. Is this because we get a product of unique functions, I do not know.
 - b. We need normalization. The divisor is an easy choice of just the attention map itself.
 - c. We need queries and keys to be different functions of the input, not just one or the other and they cannot be static.
 - d. The formula so far is

$$\exp(\text{queries.unsqueeze(-2)} * \text{keys.unsqueeze(-3)}).prod(-1)$$
 which is the same as

$$\exp(\text{queries.unsqueeze(-2)} * \text{keys.unsqueeze(-3)}.sum(-1)) = \exp(Q @ K.T)$$
- 14. Does a multiplicative relationship with the value work?

Let's examine the taylor expansion of a llama 2 model:

- We find that the distribution values $> 1e-5$ for terms (where 0 is just 1.0 and 1 is x , the linear term) is quite diverse, meaning attention uses a very nonlinear function:



- For a random initialization, we would just see 1 and 2 (linear and quadratic) being used
- One thing to note is that the distribution is very similar across layers > 5 .

We now have two pieces of evidence suggesting that RNNs cannot be as expressive as softmax attention:

1. Decomposition of the exponential
 - a. For each query and key, we have $A = e^{QK} = e^{Q_1K_1} * e^{Q_2K_2} * \dots$
let $e^x = f(x)$
Then $A = f(QK) = f(Q_1K_1) * f(Q_2K_2) * \dots$ which cannot be decomposed
However if we make this linear, we have either
 $A = f(Q_1) * f(K_1) * f(Q_2) * f(K_2) * \dots$
OR
 $A = f(Q_1 * K_1 * Q_2 * K_2 * \dots)$
which are both RNNs and bad.
2. The Taylor series has a lot of important values > 1 meaning the function is highly nonlinear

The attention numerator looks like the following:

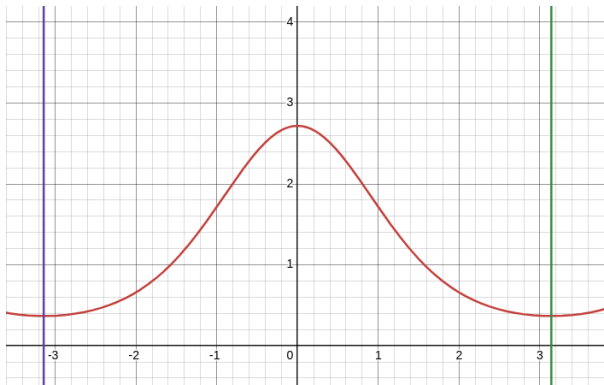
$$e^{ab \cos(x)}, a = ||q_s||, b = ||k_s||$$

- a is the magnitude of the query vector
- b is the magnitude of the key vector
- x is the angle between vector q and k

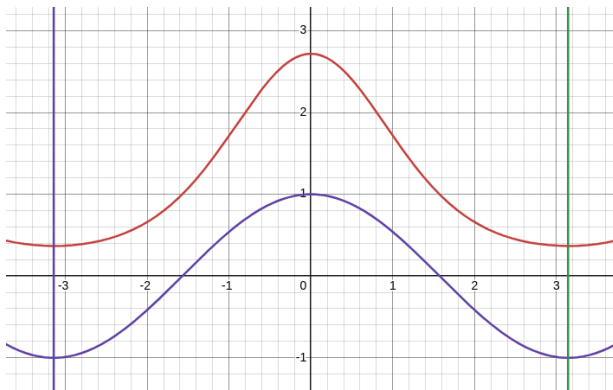
Now if we let a and b be 1 (unit vectors), we get the following:

$$e^{\cos(x)}$$

Which is just the exponentiated form of cosine attention.



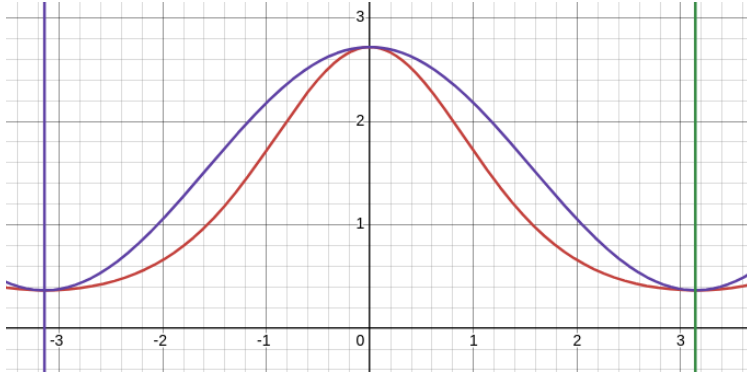
The first question I had is do we just need the functional form to match?



(cosine in purple, exp(cos) in red)

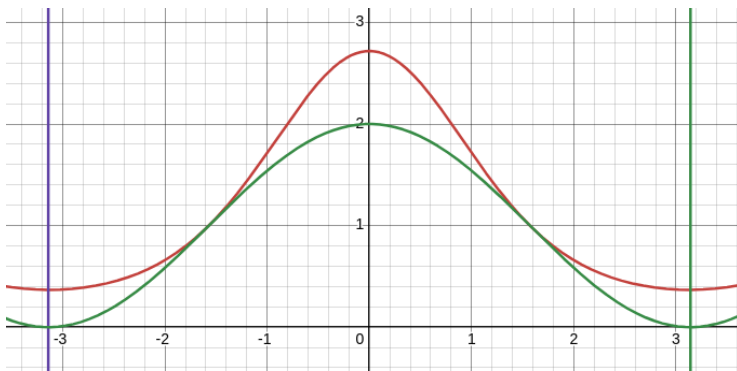
We can obviously do this by moving the cosine function up and scaling properly. The following equation does this:

$$\left(\frac{e - \frac{1}{e}}{2} \right) \cdot (ab \cos(x)) + \left(e - \frac{e - \frac{1}{e}}{2} \right) a b$$



Alternatively we can use the following equation if we something simple, a nonnegative form:

$$(ab \cos(x)) + ab$$



However both these result in garbage

This leave the question, does $\exp(\cos(x))$ work? Or is the magnitude needed?

1. If $\exp(\cos(x))$ works, then exp is likely the key. **(This is not the case!!)**

a. I am guessing it is due to the gradient.

The gradient of softmax is $-\sin(x)e^{\cos(x)}$

The gradient of cosine attention is $-\sin(x)$

Where x is the sangle between the key and query vector.

These look very similar

b. However if we instead look at this in terms of inner product, we see:

The gradient of softmax is $d/dq = ke^{q^*y^k}$

The gradient of cosine similarity is $d/dq = k$

The cosine gradient is obviously linear

Can we construct a linear function with a nice gradient that is still linear?

c. For example, if we have $(x^*y)^2$, this would give $2y(x^*y)$

2. If $\exp(\cos(x))$ does not work, then perhaps the magnitude is important, which cosine similarity does not use.

Current Remarks:

1. Exponential is needed
 - a. This is seen as using different functions causes the loss to decrease much slower
2. Magnitude is needed
 - a. Cosine attention and normalized softmax attention ($\exp(\cos(qk))$) seem to be rather equivalent (for tests at early loss), but these converge slower than normal softmax attention. Thus the magnitude is needed
3. The properties of a distribution (values between 0 and 1 and add up to 1) are very computationally nice.
 - a. Values between 0 and 1 is nice for computational stability, however this isn't needed
 - i. However, since we normalize by the row sums, having negative values can lead to massive values. For example, a value of -100 and 101 still add up to 1 even though they are massive in size while sizes this large cannot happen when only positive.
 - b. Add up to 1 is nice to prevent internal variance shift

Current questions:

1. Which part needs to be exponentiated? The magnitude or the angle part or both?
 - a. Is the cosine part the only one that needs exponentiating? $\|q\| * \|k\| * e^{\cos(qk)}$
 - i. This works the exact same as normalized softmax attention. So bad.
 - b. Is the magnitude part the only one that needs exponentiating?
$$e^{\|q\| * \|k\| - 1} * (\cos(qk) + 1)$$
 - i. Yes! This works better than normal cosine attention!!
 - c. Can we remove the magnitude and have cosine be above 0?
$$\|q\| * \|k\| * \cos(qk) + \|q\| * \|k\|$$
 - i. No, this is just as bad as cosine attention
 - d. Does it work if we instead exponential the addition too?
$$e^{\|q\| * \|k\| - 1} * \cos(qk) + e^{\|q\| * \|k\| - 1}$$
 - i. This is just as good as adding 1 to cosine
 - e. Can the exponential be decoupled? $e^{\|q\|} e^{\|k\|} e^{-1} * (\cos(qk) + 1)$
 - i. No this works as well as cosine attention
2. We know that the magnitude is needed. However in what form is this needed?

What if we add the magnitude to cosine?

$$O_t = \frac{\sum_{s=0}^T \|q_t\| * \|k_s\| * \cos(q_t k_s) * v}{\sum_{s=0}^T \|q_t\| * \|k_s\|} \quad \leftarrow \text{This works quite well}$$

Let's define the current ordering from best to worst:

1. Softmax attention
2. Multiply by the exponentiated norm $e^{\|q\| * \|k\| - 1} * (\cos(qk) + 1)$
3. a decoupled exponentiation $e^{\|q\|} e^{\|k\|} e^{-1} * (\cos(qk) + 1) \sim$ no exponentiation
 $\|q\| \|k\| * (\cos(qk) + 1)$
4. Cosine attention
5. Normalized softmax attention

Adding magnitude helps a little for cosine, but not by a ton.

More tests:

- What if we just have the magnitude?
 - $\|q\| * \|k\|$
 - This is bad as expected. Even if you exponentiate it
- What is needed for the exponential
 - As stated earlier, it is bad if we just exponentiate the $\cos(\text{angle})$
 - What if we move the magnitudes outside the exponential?
 - $\|q\| * \|k\| * e^{\cos(x)}$
 - This does not do well
 - What if the $\cos(x)$ part is changed from being between $[-1, 1]$ to $[0, 2]$, that is the input of the exponential is only positive. Or if we make it only negative by subtracting 1.
 - $e^{\|q\| * \|k\| * (\cos(x) + 1)}$ or $e^{\|q\| * \|k\| * (\cos(x) - 1)}$
 - **This works!** This means that it's not the negative/positive hypothesis.

Where am I currently:

- I now know that every part of softmax is needed, but not in the way I once thought. I thought that the $\cos(x)$ part caused the model to choose between very small values ($\cos(x) = -1$) and very large values ($\cos(x) = 1$), then amplifying this with the magnitudes. However, this is wrong since we can have strictly $\cos(x) > 0$ or $\cos(x) < 0$ and it works just the same.
- However, both the magnitudes ($\|a\| \|b\|$) and the angle ($\cos(x)$) are needed meaning both the angle and magnitudes are needed.

What are my current thoughts?

1. Since the range of $\cos(x)$ does not matter, perhaps the “relativity” part of softmax is what gives it the nice properties
 - a. To test this, can we divide by something other than the sum of the exponentials?
2. I am interpreting the $\cos(x)$ part as a kind of “slope” for the e^x and the magnitude as the value along said slope (plot $y = e^{(x \cdot k)}$) where k represents the cosine of the angle (between -1 and 1) and $x > 0$ represents the possible values of the magnitude. Essentially, the slope is controlled by the angle and the magnitude is just a value along said slope.
3. Perhaps the slope of $e^{qk} = q \cdot e^{qk}$ is what gives it its nice properties?

Is it a property of the softmax function in general? TO test this, let's move around the softmax:

Normal: $O = \text{SM}(Q K^T) V$

softmax d_V : $O = Q \text{ SM}(K^T V, \text{dim}=-1)$ over the embedding dimension (d_V)

softmax S_K : $O = Q \text{ SM}(K^T V, \text{dim}=-2)$ over the sequence dimension (S_K)

- Both of these are about as good as cosine attention!!
- Note that both of these can be converted into RNNs and are linear

Thoughts:

- I believe the reason softmax attention works so well is because it's an RNN every token, not a single RNN over all tokens
- I think there are two loss manifolds which converge to very similar spots, an RNN manifold and an RNN^2 manifold where the first is an RNN over the entire sequence and the second is a unique RNN for each token in the sequence.

What if we have a transformation between each hidden state. Which manifold does this fall on?

$$H_{t+1} = H_t @ W + K^T V$$

Running magnitude tests:

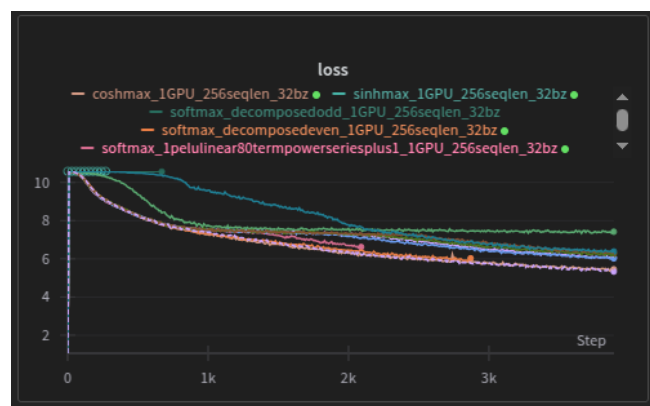
1. Normal softmax good
2. Can we normalize and remove magnitudes
 - a. No, it's complete garbage
3. What if we clamp between -2 and 2 before the exponential?
 - a. No, it's exactly the same as normalization
4. What if I clamp between -10 and 10?
 - a. Somehow this is just as bad as (-2, 2)?
5. Can I divide the logits by 100?
 - a. No this doesn't work
6. What if I don't scale by $1/\sqrt{d}$?
 - a. It still works
7. What about a Gaussian?
 - a. that is, what if I do $\text{attn_scores} = \exp(-(\text{const} * QK)^2)$
 - b. Either divisor (static, divide by S or dynamic, divide by the row sum) results in the same path as cosine attn.
 - c. What if we remove the squared term? $\text{attn_scores} = \exp(-(\text{const} * QK))$
 - i. Interestingly this doesn't work. Is it because of the negative or the learned scalar?
 - ii. It looks like the constant was the problem
 - d. Does just a negative work? $\text{attn_scores} = \exp(-(QK))$
 - i. Yes
 - e. Does a gaussian without a constant work? $\text{attn_scores} = \exp(-(QK)**2)$
 - i. No this does not
 - f. What if we subtract by 1 so that a low similarity has a zero value instead $\text{attn_scores} = 1 - \exp(-(QK)**2)$
 - i. This does not work
 - g. What if we just square it? $\text{attn_scores} = \exp((QK)**2)$
 - i. Values diverge. May or may not work
 - h. Since we cannot test squaring the inner product, let's test the absolute value: $\text{attn_scores} = \exp(\text{abs}(QK))$
 - i. This does work!!
 - i. What if we take the negative of this? Kind of like an absolute value Gaussian. $\text{attn_scores} = \exp(-\text{abs}(QK))$
 - i. This does not work
 - j. **Conclusions: Symmetry is fine, but the negative without symmetry is what makes the model not converge well. I am guessing this is the same thing with ReLU vs. sigmoid. The gradient can become small. A negative by itself doesn't do much to the gradient.**
8. Again, what if I test a different base like 1.1? $\text{attn_scores} = 1.1**(QK)$
 - a. This works as expected, but the values are massive. Like in the hundreds.
9. What if we hope that the values don't explode and do a learned denominator of the sequence length: $(1.1**(QK)) / (\text{seq_len}**\text{const})$
 - a. **This does not work. This means the relativistic nature of softmax is needed**

10. Conclusions:

- a. Since negating the exponential results in problems, we know that we need a nice and large gradient which we don't have with the negative exponential.
- b. Squaring and absolute value with symmetry is fine as long as the gradient is nice. making these negative gives a small gradient
- c. The learned denominator is needed for softmax to work. It doesn't matter for linear attention, but it does matter for softmax attention. This is needed to create relativistic values. Important values will be massive while non important values will be small. Without this, softmax doesn't work because it doesn't compare the values.

Some tests:

1. What happens if we have a stop grad for the denominator:
 - a. It does worse. Somewhere in between normal softmax and linear attention.
2. What happens if we have a learnable scalar for each part in the Taylor expansion?
 - a. The model diverges
3. What happens if I remove the first term (+1) from the Taylor series
 - a. It does absolutely terrible
4. What happens when I use the good ol
 - a. $A = (1 + \text{elu}(q)) @ (1 + \text{elu}(k))$.
 - b. It does as well as cosine attention
5. Now what happens if I square the inner product?
 - a. $A = ((1 + \text{elu}(q)) @ (1 + \text{elu}(k)))^2$
 - b. It's a little better than (4)
6. Now what happens if I do a Taylor series to 4 terms?
 - a. $I = (1 + \text{elu}(q)) @ (1 + \text{elu}(k))$
 - b. $A = I + I^2/2 + I^3/6 + I^4/24$
 - c. It's a little better than (5)
7. What happens if I do the Taylor series to 80 terms?
 - a. It's even closer to softmax



- b.
 - c. Softmax is purple, normal 1+elu is blue, 80 terms is red. We can see as we add more terms, it approaches softmax.
8. Does the same behavior happen for ReLU?
 - a. Yes, if I do terms from 1-->80, we same the exact same pattern
 - b. Interestingly, ReLU does worse than cosine attention normally, but when we get to 80 terms, it does better than cosine attention
9. Does the same behavior happen to Cosine attention without the sum denominator?
 - a. No, adding more terms does nothing.
10. What if I just do even terms for the series of exp (cosh)?
 - a. As expected, this is almost as good as softmax, but not quite. This is because only half the Taylor terms are used
11. What if I just do odd terms for the series of exp (sinh)?
 - a. This is and diverges because it has negatives
12. What if I do $A = \cos(qk) + \sin(qk)$?

- a. This is ok. It's in between (5) and (6), so maybe cubed?
- 13. What if I don't have an activation and just subtract the min?
 - a. This is terrible
- 14. Can I divide by the variance instead of the sum of coefficients?
 - a. No, this doesn't work. Maybe cause values get massive.
- 15. Does L2 distance instead of inner product work?
 - a. $A = e^{(L2_dist(q, k))}$
 - b. YES!
- 16. Does covariance work?
 - a. This seems to be converging to the same place as softmax.
- 17. If we look at similar Taylor expansions to $\exp$, $1/(1 \pm x)$ looks very similar.
 - a. The only difference is the exponential is divided/bounded by the exponential
 - b. This has a max of 1 or min of -1.
 - c. This does not work as well, likely due to the required domain clipping
- 18. Let's try to remove the denominator. What if we replace it with $1/S$?
 - a. This does not work!
- 19. What if we replace the denominator with $1/S$, but gate the attention weights: $A = \exp(Q@K) * \text{gate}$ where gate is applied row-wise to each key or $A = \exp(Q@(K*\text{gate}))$
 - a. This does work!
 - b. However it is unstable at a point as values get large
 - c. Claim: The denominator has two uses
 - i. It allows for spiky weights. That is we can have weights be close to zero and some be massive like 10000000. The denominator allows these values to exist. Without the denominator, it would be unstable for massive values to exist, thus causing instability. With the denominator, we can have massive values and therefore create spiky weights.
 - ii. It allows for long sequences while not decaying the past. We can continuously add more terms, but unlike an RNN the weights are flattened rather than decaying the past.
 - d. What about just $1/S$?
 - i. This does not allow for massive values
- 20. Now that we know What if we replace the denominator with $1/S$, but gate the attention weights: $A = \exp(Q@K) * \text{gate} * 1/S$ works, can we get it stable. Let's try limiting the inner product to have a max value of 5. Also, let's normalize the output after the values.
 - a. This works! Note that this does not work without the gate and is not as good without output gate since magnitudes get large
 - b. Do we need to $\exp$ then?
 - i. No, ReLU works!
 - ii. Doing a linear function or kernel function does not work
- 21. Ok how about the good ol trick of dividing by a learnable value of S ?
 - a. This doesn't work.
 - b. Since we are having a max value of 5, let's set $s = e^5 * \text{seq_len}$ instead of just $S = \text{seq_len}$
 - c. In total we have: $A = \exp(Q@K) * \text{gate} * 1/(e^5 * \text{seq_len})$ without an output norm

- i. This works! But it's about as good as just the output norm
 - d. It also works if we have a learnable power of the seq_len $A = \exp(Q@K) * \text{gate} * 1/(e^5 * \text{seq_len}^{**C})$
22. Ok we know we need to clamp the exponential so it doesn't get too large. However, doing so requires either dividing by S and having a norm or dividing by an upper bound with no norm (and possible learnable power) what if we just divide by the upper bound and have an output norm?

Conclusions:

1. It seems a nice exponentiated distance function is all you need
2. This means that the exponential is the important part, not the choice of distance function.
 - a. This does give us an infinite dimensional inner product
3. Instead of dividing by the sum of weights, we can instead add a gate and divide by S
 - a. Note that without the gate this does not work
 - i. The reason for this is the gradient is no longer all-to-all. With the gate, it is all-to-all
 - b. The divided by S and out norm are needed to keep the magnitude from getting too large

What do we need:

1. Exponential on inner product space (numerator)
 - a. To test this is easy. Just convert this to a linear kernel like $\text{elu}(x)+1$
- ~~2. All-to-all gradient (denominator)~~
 - ~~a. To test this, first we can show that by detachign the denominator we don't get as good of accuracy~~
 - ~~b. We can then add a different function that does give the model this all-to-all gradient communication (that is, we need~~
 - c. This seems to be wrong as we can apply a pre-gate
3. (alternative to 2) It seems like we just need a learnable modulation/gate for every value.
 - a. For softmax, this is a gate that is unique for each query and the same for every key. This value is learnable and happens to be nice as it's the sum of
 - i. This is a gate right before the output. That is, once the hidden state has been updated, we then gate the output by the sum of all the keys multiplied by the query then we multiply by the query.
 - b. Trials above have been using a gate unique to each key, not unique to each query.
 - i. This is a gate on $K^T V$ before adding to the hidden state unlike query-unique gate which is post hidden-state update.
 - c. So, we essentially have two gates. One between Q and K (post hidden state gate) and the other between K and V (pre hidden state gate)
 - d. This would be interesting to experiment with. However, one this remains consistent. Decaying is stupid (gate on the hidden state)

- i. In the attention formulation, this is a reverse cumprod applied to a post-gate, that is the gate after the hidden state.
4. A way to keep outputs from getting too large and to have a stable algorithm (also from the denominator)

Things to test:

- Q vs K gate (can we do this with normal softmax? That is, what if we take the sum over columns?)
 - Yes this does work as well as softmax
- How does linear perform compared to exp?
 - Linear is quite bad
- Does squaring work as opposed to exp?
 - Squaring is much better than linear, but not quite there
- Does cubing work?
 - Cubing is a little better than squaring, getting a little closer to exp but not quite there
- Does tanh gate work?
 - I think in the limit it will, but it's very noisy and quite bad
- Take the Triton kernel for flash attn. Repurpose for gated attn just on the values w^w

Mamba Sutf:

Seems like there are the following major parts to mamba over linear attention:

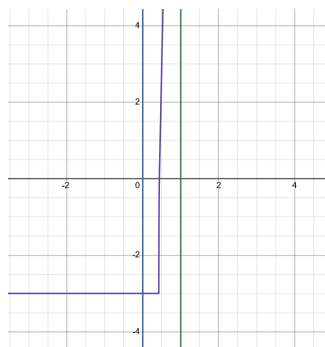
1. Input time-based convolution on Q, K, and V
2. SiLU activation function on Q, K, and V after the convolution
3. A discretization parameter (dt) which is similar to Q,K,V as it is a projection from the input, though instead of being projected to $H*d$ it is just H
 - a. Applied to the values via Hadamard product
 - b. Creates the A mask by multiplying dt by the A values.
4. Learnable values of A of shape (H) as noted above.
5. A decay by exponentiated cumulative sums of the A values.
6. Output residual (D residual) which comes from a learnable D param of shape (H) or $(d*H)$ then multiplied by the values: $D_residual = D*values$
7. Output gate which comes from an additional projection of the input to a value called z. This gate is like SwiGlu and is applied after the residual: $y = y * SiLU(z)$
8. Output norm which can either have statistics from z or can be an RMSNorm
9. Output projection

What is actually needed?

- Seems like the following three are needed to get this thing working well
 - a. Input convolution
 - b. A decay mask
 - c. Output norm (RMSNorm works)

What's up with this dt, A mask thing?

- The dt bias is formed as follows:
 - $dt = \exp[\text{rand}(H, \text{min}=0, \text{max}=\log(0.1) - \log(0.001))] + \log(0.001)$
 $= \exp[\text{rand}(H, \text{min}=0, \text{max}=4.6)] - 7$
 - Wtf is this lol
 - $dt = dt.\text{clamp}(\text{min}=1e-4)$
 - $dt_bias = dt + \log(1 - \exp(-dt))$
 - I think this is what it looks like:



- A is just a parameter in the log domain between 0 and 16:
 $A_log = \log(\text{rand}(H, \text{min}=1, \text{max}=16))$

- The forward pass produces dt as follows:
 - $dt = \text{proj}(x)$ is of shape (H)
 - $dt = \text{softplus}(dt + dt_bias)$
 - The dt bias seems to give the value either a value of 0 or a super sharp increase. Without it, you just have normal softplus with a small increase.
- Finally, A is created from the following:
 - $A = -\exp(A_log)$
 - This just takes it out of the log domain
 - $A = A * dt$
 - This seems to multiply the value of dt by a value between 1 and 16
- Then the mask is created by doing exp then cumulative multiplications or, more effectively, cumsums then exponentiation.

Questions:

1. Is conv only needed on Q, K, or V?
 - a. Seems like a conv on only Q is slightly better than a conv on K and V, but not by a ton.
2. Is norm needed?
 - a. No, but it does better with it
 - b. Seems like the A mask already handles large values due to the decay
3. Is dt needed on the values?
 - a. Surprisingly yes
4. Does conv(4) do better than conv(2)
 - a. Seems like conv(2) is slightly better
5. Is the bias needed?
 - a. Yes, the model is significantly worse without it
6. Is the A necessary?
 - a. Without A, the model is slightly worse. Maybe due to the initialization of the bias
7. What if RoPE is used instead of A?
 - a. This does as good as if we didn't have the A matrix at all?
8. Does A need to be in the log domain?
 - a. It seems like the model initially does fine, but then as it goes on, the model increases in loss. It doesn't die, just gets bad for some reason.
https://wandb.ai/gmongaras1/Gated_Attention_V2/runs/bt0gn46e
9. Does the dt bias need to be initialized like whatever tf it's initialized as?
 - a. IDK let's try initialization between 0 and 1 then exp.
 - b. If the initialization is between 0 and 1, it's bad
 - c. If the initialization is between -1 and 0, it's better
 - i. Exp or softplus doesn't matter
10. Let's look at the definition of dt and A again:
 - a. Initialization:

- i. $dt = \exp(\text{uniform}[H; \min=\log(0.001), \max=\log(0.1)])$ is a value between $[0.001, 0.1]$
 - ii. $dt_inv = dt_bias = dt + \log(1 - \exp(-dt))$ is a value between $[\log(0.001), \log(0.1)] = [-6.9, -2.3]$
 - 1. This is essentially the inverse of softplus. That is, the model is optimizing in the pre-softplus domain. It is initialized such that $\text{softplus}(dt_inv)$ is a value between $[0.001, 0.1]$
 - 2. This is the value the model optimizes.
 - 3. So we basically initialize values between 0.001 and 0.1 post-softplus, just with a weird distribution.
 - iii. $A_log = \log(U[H; \min=1, \max=16])$
 - 1. The model learns some value between 1 and 16, but in the log domain.
- b. Forward pass
- i. $dt = \text{softplus}(dt_proj(x) + dt_bias)$ is a value between $[0, \infty)$
 - 1. If the projection is 0, then this is a value between $[0.001, 0.1]$ as this is how the bias is initialized
 - ii. $V = V * dt$
 - 1.
 - iii. $A = -\exp(A_log)$
 - 1. This is a value between $[-1, -16]$, but optimized in the log domain
 - iv. $A = A*dt$ is a value between $(-\infty, 0]$
 - 1. when the projection is 0, A is a value between $[-16*0.1, -1*0.001] = [-1.6, -0.001]$
- c. Can we initialize dt_bias to just be a value between 0.001 and 0.1 and be added to the post-softplus value of the projection:
- $dt = \text{softplus}(dt_proj(x) +$

Back to attn:

- If exp on d_k works so well, what about exp on the t dim or exp on the d_v dim?
 - exp on either of these results in linear attn
- What if instead of doing $\sum_s e^{Q_t K_s} V_s$
We instead do $e^{Q_t \sum_s K_s V_s} = e^{Q_t K V}$
 - This does not work! It gives linear attention accuracy. Essentially it turns the hidden state from additive over time to multiplicative over time.
 - Also, this makes both d_k and t infinite dimensional. This means that infinite t does not work.
 - https://wandb.ai/gmongaras1/Gated_Attention_V2/runs/s5w899o0
- For a Taylor expansion, we have n term “weights” via dividing by the factorial. Can we learn these values
 - Method 1: Use the gamma function
 - The model eventually diverges for some reason. Maybe because the weights get too big?
 - Seems that fractional powers suck
 - Method 2: Learn n terms where each term has both the weight ($1/i!$) and the power learned. Note that for this we use the gamma function to calculate a fractional $i!$ and complex numbers to learn the fractional power.
 - The model also diverges interestingly
 - Seems like fractional powers suck?
 - Method 3: Keep the powers of n and instead learn a sigmoid (multiplied by 2 and initialized to 0. Thus initializing post sigmoid to 1).
 - This seems to work well.
 - Looking at the order coefficients, it seems like odd powers are preferred over even powers??

```
> torch.stack([(self.model.module.model.layers[i].self_attn.order_co
> torch.stack([(self.model.module.model.layers[i].self_attn.order_co
> [(self.model.module.model.layers[i].self_attn.order_coefficients.s
> special variables
> function variables
> 00 = tensor([0.9788, 1.0565, 0.8356, 1.1628, 0.8270], device='cu...
> 01 = tensor([0.9521, 1.0554, 0.8772, 1.1307, 0.9514], device='cu...
> 02 = tensor([0.9457, 1.0535, 0.8703, 1.1265, 0.9464], device='cu...
> 03 = tensor([0.9358, 1.0631, 0.8781, 1.1201, 0.9443], device='cu...
> 04 = tensor([0.9613, 1.0426, 0.9067, 1.0938, 0.9689], device='cu...
> 05 = tensor([0.9602, 1.0356, 0.8966, 1.0997, 0.9650], device='cu...
> 06 = tensor([0.9788, 1.0135, 0.9380, 1.0633, 0.9899], device='cu...
> CALL STACK
```

- Values got more extreme as training went on. Like to values of 0.8 and 0.6.
- Let's see how good a fourth order model works (with sigmoidal gating):
 - It seems work work ever so slightly worse than softmax!
 - What if we only have the fourth order term?
 - It seems like the model is slightly worse, but almost as good as full fourth order

- What if we add a time convolution?
 - Interestingly, it does slightly worse?
- If the above shows that odd powers are more important, can we only use odd powers?
 - Does powers 1 and 3 work?
 - Looks like it's not really working too much.
 - Do powers 0 (bias), 1, and 3 work?
 - Yep, looks like it needs a bias term (initialized to 1)
- Does linear work?
 - Looks like simply adding a learnable "0 power" removes the issue of it being slow at the beginning of training.
 - Adding an in conv also works to speed up the beginning of training to follow the "good path"
 - It's slightly better with the conv, but the main thing seems to be this learnable bias term. Instead of just a linear projection, making it an affine projection with a bias.
- What about for 6 terms?

```

√ [(self.model.module.layers[i].self_attn.order_coefficients.sigmoid(
> special variables
> function variables
> 00 = tensor([0.9900, 1.0371, 0.8985, 1.1093, 0.8591, 1.1668, 0.8398],
> 01 = tensor([0.9976, 1.0341, 0.9594, 1.0611, 0.8857, 1.1223, 0.9461],
> 02 = tensor([0.9749, 1.0391, 0.9321, 1.0727, 0.8771, 1.1177, 0.9267],
> 03 = tensor([0.9916, 1.0195, 0.9556, 1.0391, 0.9091, 1.0890, 0.9598],
> 04 = tensor([1.0067, 1.0056, 0.9715, 1.0281, 0.9292, 1.0767, 0.9704],
> 05 = tensor([0.9982, 1.0114, 0.9666, 1.0337, 0.9261, 1.0835, 0.9650],
> 06 = tensor([0.9673, 1.0355, 0.9410, 1.0611, 0.9104, 1.0961, 0.9342],
> 07 = tensor([0.9962, 0.9851, 0.9629, 1.0105, 0.9170, 1.1000, 0.9586],
> 08 = tensor([0.9747, 1.0357, 0.9559, 1.0568, 0.9217, 1.0824, 0.9485],
> 09 = tensor([0.9756, 1.0277, 0.9645, 1.0512, 0.9455, 1.0722, 0.9608],
> 10 = tensor([0.9715, 1.0270, 0.9524, 1.0437, 0.9258, 1.0825, 0.9652],
> 11 = tensor([0.9683, 1.0317, 0.9467, 1.0586, 0.9168, 1.0987, 0.9536],
> 12 = tensor([0.9577, 1.0347, 0.9347, 1.0667, 0.8975, 1.1256, 0.9358],
> 13 = tensor([0.9381, 1.0438, 0.9259, 1.0763, 0.8995, 1.1371, 0.9311],
> 14 = tensor([0.9207, 1.0171, 0.9029, 1.0715, 0.8688, 1.1848, 0.8938],
> 15 = tensor([0.9328, 1.0439, 0.9249, 1.0737, 0.9117, 1.1223, 0.9400],
> 16 = tensor([0.9058, 1.0524, 0.9010, 1.0947, 0.8817, 1.1749, 0.9204],
> 17 = tensor([0.9134, 1.0590, 0.9101, 1.0897, 0.9006, 1.1403, 0.9389],
> 18 = tensor([0.9094, 1.0650, 0.9123, 1.0962, 0.9018, 1.1407, 0.9136],
> 19 = tensor([0.9390, 1.0483, 0.9322, 1.0646, 0.9253, 1.1004, 0.9530],
len() = 20

```

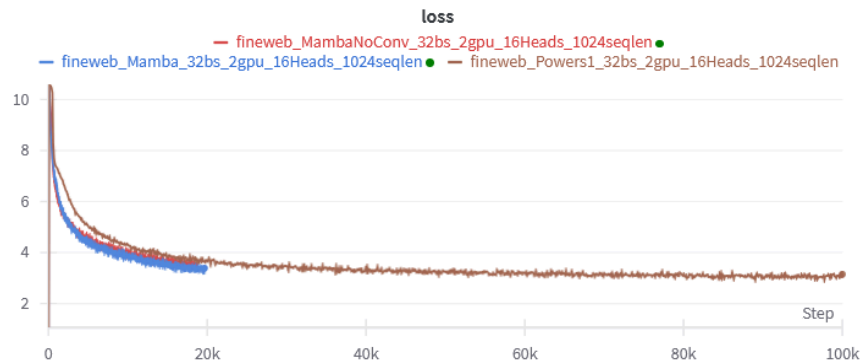
- Looks like the same thing. The odd terms matter way more
- What if we do $\sinh(qk) + 1$?
 - This works almost as well as pure softmax and on the last step it's better!

Loss curve where each plot show the normal increasing powers plot and there are multiple plots over varying head dimensions.

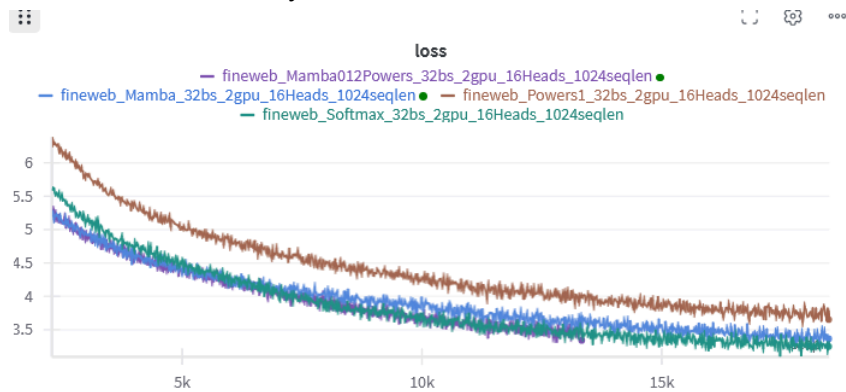
Seems like making the dimension larger barely effects exp but does seem to make linear attention better.

Now that I fixed mamba, I can test it:

- Does removing the in conv work?
 - No, it is significantly worse. Near linear attention, but still better



- What about powers 0,1,3?
 - It's better and basically matches softmax



- What about powers 0,1,2?
 - ...
- Can we remove the D residual?
- Can we remove the Z gate?
- NOTE: It looks like dt is a low rank way of adding heads which is probably why it's needed.

Tests (using RoPE):

- ☒ P1
- ☒ P01
- ☒ P01 + learnable
- ☒ P01 + low rank heads
- ☒ P01 + learnable + low rank heads
- ☒ P1 + conv
- ☒ P01 + conv
- ☐ P01 + conv + learnable + low rank heads
- ☒ P2
- ☒ P012
- ☐ ...
- ☒ Softmax

Tests (no Rope - NoPE)

- ☒ P1 + NoPE
- ☒ P01 + NoPE
- ☒ P01 + NoPE + learnable
- ☒ P1 + NoPE + conv
- ☒ P01 + NoPE + conv
- ☒ P2 + NoPE
- ☒ P012 + NoPE
- ☐ P1 + NoPE + AMask
- ☐ ...
- ☒ Softmax + NoPE

Notes:

- Looks like low rank heads does worse than full rank heads. I guess this is to be expected. One thing a friend told me is that GQA actually breaks down at scale, so something to keep in mind.
- P1 + NoPE + Conv + AMask is almost at Mamba
- P2 + NoPE + Conv + AMask is better and almost at softmax

It looks like conv and no positional encodings and AMask is almost there. But what makes mamba actually good? There are two additional things I need to add:

1. values discretizations: $V = V * dt$
2. dt bias

$V * dt$ is basically $V * \text{softplus}(x)$ which is similar to silu, except linear in the slope as opposed to exponential.

- Is this just an activation function on V like SiLU?
- Looking at the formula, is this a "learning rate" value that is actually needed?
- Is softplus necessary? This is strictly positive.

A is created as $-\exp(A_log)*dt$. Is this necessary or can $-\text{softplus}$ just be used? Note that this value cannot be positive, so silu or $x*\text{softplus}(x)$ cannot be used.

Tests - What makes mamba good?

- ☒ ~~Normal (dt on A, dt on V)~~
- ☒ ~~softplus on A, dt on V~~
- ☒ ~~dt on A, silu on V~~
- ☒ ~~dt on A, softplus on V~~
- ☒ ~~dt on A, $x*\text{softplus}(x)$ on V~~
- ☐ dt on A, $V*\text{softplus}(\text{learnable value})$

Notes:

- It looks like softplus of a learnable A projection is just as good as coupling A and dt like mamba does it.
- Other activations on V do not work. This means we need a separate time discretization or learning rate parameter on the values. This value should be strictly positive

Stuff to test

- Rank of softmax attn and mamba
- How does the model do if I just double the state? If I quadruple it? That is, let's take the reduced version of linear mamba and show squaring is the same as using a hidden state size of about 2000
- How does the square model do with a hidden state size of 32 (double # of heads). What about 128 (half number of heads)